



DOCUMENTO TÉCNICO DE PRODUCTO

Arquitectura de Software

Marketplace conversacional, pagos y última milla

DECISIÓN ARQUITECTÓNICA

Comenzar como un monolito modular orientado a eventos, desplegable en contenedores y preparado para separar servicios sólo cuando el volumen, el equipo o el riesgo operativo lo justifiquen.

Versión 1.0 · Arquitectura objetivo del MVP · Julio 2026

Control del documento

Campo	Definición
Propósito	Definir una arquitectura implementable, segura y evolutiva para el MVP de Livra.
Audiencia	Dirección, producto, arquitectura, desarrollo, DevOps, seguridad y proveedores.
Alcance	Pedidos por WhatsApp, catálogo, pagos, despacho, tracking, entrega y liquidaciones.
Fuera de alcance	Estimaciones de esfuerzo cerradas, presupuesto contractual y diseño detallado de UI.
Estado	Propuesta técnica sujeta a validación con Meta, Mercado Pago y operación piloto.

Mapa de contenidos

- 1. Objetivos y principios
- 2. Contexto y alcance
- 3. Arquitectura lógica
- 4. Componentes y responsabilidades
- 5. Flujos críticos
- 6. Modelo de datos
- 7. Integraciones externas
- 8. Seguridad y privacidad
- 9. Despliegue y DevOps
- 10. Observabilidad y operación
- 11. Escalabilidad y resiliencia
- 12. Estrategia de pruebas
- 13. MVP técnico y roadmap
- 14. Decisiones pendientes
- Anexos técnicos

1. Objetivos y principios

La arquitectura debe permitir que un cliente complete el ciclo comercial sin instalar una aplicación; que el restaurante opere desde WhatsApp o un panel liviano; y que el repartidor reciba, acepte y ejecute viajes mediante una PWA. El sistema debe conservar trazabilidad completa desde la conversación hasta la liquidación.

Principios rectores

- Canal desacoplado del dominio: WhatsApp es una interfaz, no la fuente de verdad del pedido.
- Idempotencia en pagos, webhooks y cambios de estado.
- Privacidad por diseño para ubicación, direcciones y datos personales.
- Operación observable: cada pedido debe poder explicarse de punta a punta.
- Automatización gradual con capacidad de intervención humana.
- Modularidad antes que microservicios; simplicidad operativa antes que sofisticación.

CRITERIO
La arquitectura optimiza primero por velocidad de aprendizaje y confiabilidad operacional; no por escala hipotética.

2. Contexto, actores y alcance

2.1 Actores del sistema

Actor	Interacción principal	Canal
Cliente	Descubre, arma carrito, paga, sigue y valida la entrega.	WhatsApp + enlace web
Restaurante	Publica catálogo, acepta, prepara y marca listo.	WhatsApp + panel web
Repartidor	Acepta viaje, navega, comparte posición y confirma entrega.	PWA móvil
Operaciones Livra	Supervisa excepciones, soporte, conciliación y configuración.	Backoffice web
Administrador	Gestiona comercios, planes, permisos e integraciones.	Consola administrativa

2.2 Capacidades incluidas

- Onboarding y configuración multi-restaurante.
- Catálogo normalizado y disponibilidad por sucursal.
- Conversación, carrito, checkout y máquina de estados del pedido.
- Pago online, webhooks, conciliación y registro de saldos.
- Despacho, tracking temporal y prueba de entrega por código.
- Notificaciones, soporte, auditoría y métricas.

3. Arquitectura lógica

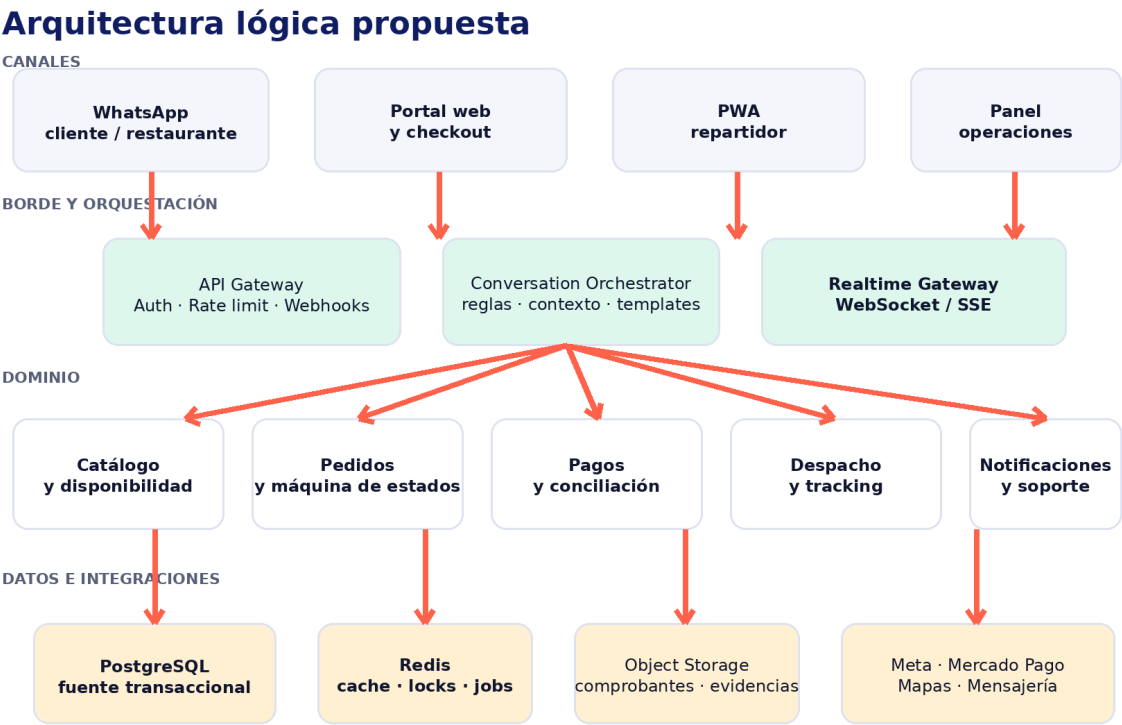


Figura 1. Capas y módulos principales de Livra.

La propuesta separa canales, borde, dominio e infraestructura. Las interfaces pueden evolucionar sin duplicar reglas de negocio. El backend conserva la autoridad sobre carrito, importe, estado, pago, asignación, posición aceptada y evidencia de entrega.

3.1 Estilo arquitectónico

Para el MVP se recomienda un monolito modular con límites explícitos entre dominios y publicación de eventos internos. Los workers asíncronos procesan tareas demoradas o reintentables. Esta estructura reduce costo de despliegue y debugging, manteniendo una ruta clara hacia servicios independientes.

3.2 Comunicación

Tipo	Uso	Regla
HTTP/JSON	Comandos y consultas de UI e integraciones.	Versionado, autenticación e idempotency key.
Webhooks	Eventos de Meta y pasarela de pagos.	Verificación de firma, persistencia y ACK rápido.

Tipo	Uso	Regla
Eventos internos	Propagación entre módulos y workers.	Outbox transaccional y consumidores idempotentes.
WebSocket / SSE	Tracking y actualización de estados.	Sesiones temporales y fallback por polling.

4. Componentes y responsabilidades

4.1 Capa de experiencia

- Adaptador WhatsApp: recibe webhooks, normaliza mensajes y envía respuestas/templates.
- Portal cliente: checkout, tracking y gestión de permisos mediante enlaces firmados y temporales.
- PWA repartidor: ofertas, navegación externa, geolocalización, hitos y código de entrega.
- Panel restaurante: catálogo, horarios, stock, aceptación, preparación e incidencias.
- Backoffice: búsqueda global, timeline, soporte, conciliación y configuración.

4.2 Servicios de dominio

Módulo	Responsabilidad	Entidades principales
Identidad y tenancy	Usuarios, roles, sucursales, sesiones y permisos.	Tenant, User, Role, Session
Catálogo	Productos, variantes, precios, horarios y disponibilidad.	Catalog, Product, Variant, Availability
Conversación	Contexto, intención, carrito y handoff humano.	Conversation, Message, Cart
Pedidos	Totales, estados, reglas, SLA e incidencias.	Order, OrderItem, OrderEvent
Pagos	Checkout, webhooks, reembolsos y conciliación.	Payment, Refund, LedgerEntry
Logística	Cotización, oferta, asignación, tracking y entrega.	Delivery, Offer, LocationPing, Proof
Notificaciones	Mensajes transaccionales y preferencias.	Template, Notification, DeliveryReceipt

5. Flujos críticos

Flujo crítico: pedido → pago → entrega

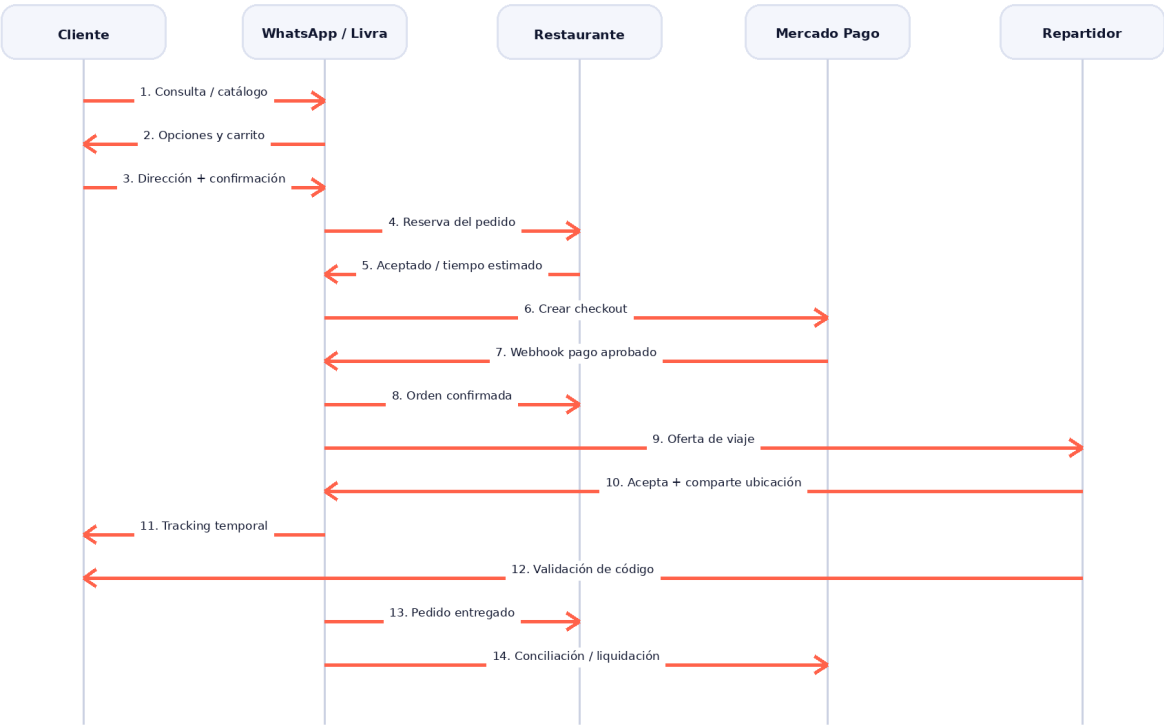


Figura 2. Secuencia nominal del pedido y la entrega.

5.1 Máquina de estados del pedido

Estado	Entrada válida	Salida / evento
DRAFT	Carrito creado.	AWAITING_PAYMENT
AWAITING_PAYMENT	Importe congelado y checkout emitido.	PAID o EXPIRED
PAID	Webhook verificado y conciliado.	ACCEPTED o REFUND_PENDING
ACCEPTED	Restaurante confirma.	PREPARING
PREPARING	Producción iniciada.	READY
READY	Pedido disponible para retiro.	PICKED_UP
PICKED_UP	Moto retira y tracking activo.	IN_TRANSIT
IN_TRANSIT	Código y reglas validados.	DELIVERED o DELIVERY_EXCEPTION
DELIVERED	Cierre irreversible del flujo nominal.	Settlement pending

INVARIANTE

El estado no se modifica directamente desde una interfaz. Todo cambio pasa por un comando validado, genera un evento auditable y aplica control de concurrencia.

6. Modelo de datos

6.1 Estrategia de persistencia

PostgreSQL funciona como fuente transaccional. Cada registro sensible pertenece a un tenant y, cuando corresponde, a una sucursal. Redis conserva caché, locks de corta duración, rate limits y trabajos; nunca reemplaza la fuente de verdad. Los archivos y evidencias se guardan en object storage con referencias en base de datos.

6.2 Relaciones esenciales

Agregado	Relaciones	Regla de consistencia
Restaurant / Branch	1:N catálogo, pedidos y usuarios.	Aislamiento por tenant_id y branch_id.
Order	N items, 1 payment intent, 0..1 delivery.	Snapshot de precio y producto al confirmar.
Payment	1:N eventos, reembolsos y asientos.	Monto y moneda inmutables tras aprobación.
Delivery	N ofertas, 0..1 repartidor activo, N pings.	Una asignación vigente por pedido.
Ledger	Entradas débito/crédito por actor.	Suma balanceada y referencias idempotentes.

6.3 Retención

- Ubicaciones: retención mínima necesaria, con precisión y frecuencia acordes al viaje.
- Mensajes: separar contenido operativo de metadatos de auditoría.
- Tokens: cifrados, rotables y nunca registrados en logs.
- Evidencias: política explícita por finalidad legal, soporte y fraude.

7. Integraciones externas

7.1 WhatsApp Business Platform

El adaptador recibe eventos mediante webhook y traduce mensajes interactivos, catálogos y estados de entrega al modelo interno. La lógica de pedido no debe depender del formato del proveedor. Los mensajes salientes se encolan, controlan por rate limit y registran con su identificador externo.

7.2 Mercado Pago

En el modelo marketplace 1:1, cada restaurante autoriza a Livra mediante OAuth. El backend crea el checkout con las credenciales del seller y registra la comisión de marketplace. El resultado definitivo proviene del webhook y de una consulta posterior cuando sea necesario; nunca del retorno del navegador.

DEPENDENCIA COMERCIAL

La arquitectura admite un ledger interno para liquidación de repartidores. La posibilidad de dividir directamente el cobro entre más de dos participantes debe confirmarse contractual y técnicamente antes de prometerla.

7.3 Mapas y geocodificación

- Geocodificación de dirección a coordenadas y normalización de domicilio.
- Cálculo de distancia/ETA y validación de zona de cobertura.
- Enlace de navegación para el repartidor; no es necesario construir navegación propia.
- Proveedor encapsulado tras una interfaz para evitar dependencia rígida.

7.4 Contrato de webhooks

Control	Implementación
Autenticidad	Verificar firma/secreto según proveedor.
Disponibilidad	Responder rápido y procesar de forma asíncrona.
Duplicados	Clave única por proveedor + event_id.
Orden	No asumir entrega ordenada; comparar versión y estado.
Reintentos	Backoff exponencial, dead-letter y replay controlado.
Auditoría	Guardar hash, cabeceras permitidas, recepción y resultado.

8. Seguridad y privacidad

8.1 Modelo de amenazas prioritario

Riesgo	Control principal
Acceso cruzado entre restaurantes	Tenant scoping obligatorio, pruebas de autorización y RLS opcional.
Webhook falsificado o repetido	Firma, timestamp, idempotencia y allowlist cuando aplique.
Robo de enlace de tracking	Token aleatorio, TTL, alcance a un pedido y revocación al finalizar.
Entrega fraudulenta	Código de un solo uso, proximidad, sesión activa y registro de excepción.
Exposición de ubicación	Minimización, consentimiento, cifrado y borrado programado.
Compromiso de credenciales	Secret manager, rotación y acceso de mínimo privilegio.

8.2 Controles de plataforma

- TLS extremo a extremo; cifrado administrado en reposo.
- RBAC por rol, tenant y sucursal; MFA para administración.
- OWASP ASVS como referencia y revisión de dependencias/SBOM.
- Logs sin secretos, datos de tarjeta ni ubicación exacta innecesaria.
- Auditoría inmutable de cambios críticos y accesos administrativos.
- Backups cifrados, restauraciones probadas y plan de respuesta a incidentes.

PAGO SEGURO

Livra no debe almacenar datos completos de tarjeta. El checkout y la tokenización quedan en la pasarela de pago.

9. Despliegue y DevOps

Vista de despliegue y operación

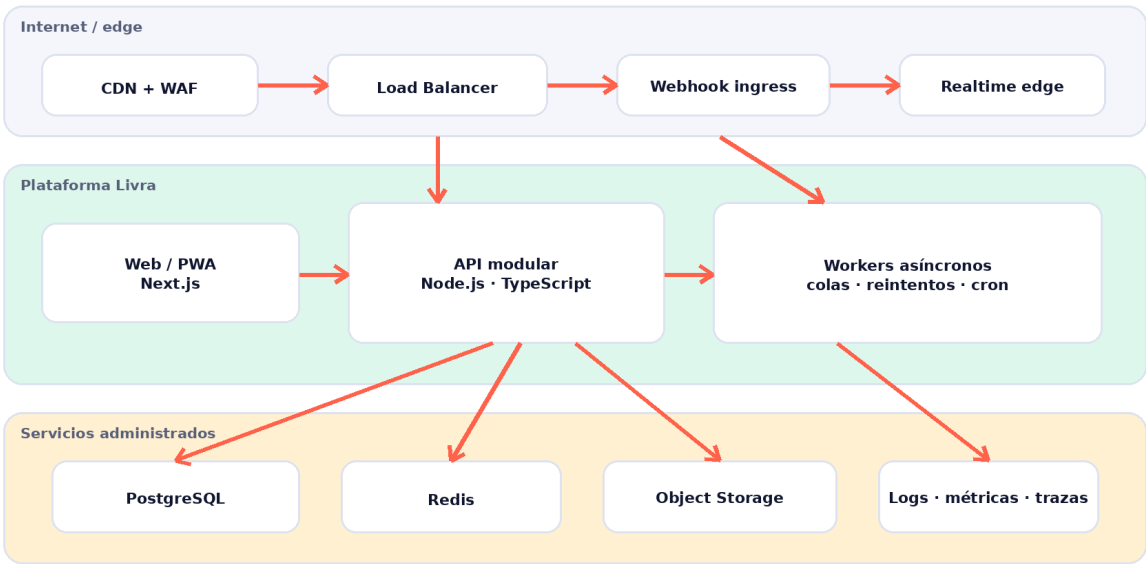


Figura 3. Despliegue recomendado para el MVP.

9.1 Stack de referencia

Capa	Opción recomendada	Alternativas
Web / PWA	Next.js + TypeScript	React/Vite para frontends separados
API	Node.js + TypeScript + NestJS/Fastify	Express con estructura modular
Datos	PostgreSQL administrado	Compatible PostgreSQL
Jobs / cache	Redis + BullMQ	Service Bus / SQS según nube
Storage	Object Storage S3-compatible	Azure Blob / GCS
Infraestructura	Contenedores administrados	Azure Container Apps, Cloud Run, ECS
Observabilidad	OpenTelemetry + plataforma gestionada	Grafana stack

9.2 Entornos y entrega

- Desarrollo local reproducible con contenedores.
- Preview por pull request para frontends.
- Staging con integraciones sandbox y datos sintéticos.

- Producción separada por cuentas/proyectos y secretos propios.
- CI con lint, tests, análisis de seguridad y migraciones verificadas.
- CD gradual con health checks, rollback y migraciones backward-compatible.

10. Observabilidad y operación

Cada pedido debe tener un `correlation_id` compartido por conversación, pago, entrega, mensajes y trabajos. La timeline operativa reúne eventos de negocio y señales técnicas sin exponer información sensible.

10.1 Señales mínimas

Tipo	Ejemplos
Métricas técnicas	Latencia, error rate, saturación, cola, reintentos y conexiones realtime.
Métricas de negocio	Conversión, pago aprobado, aceptación, preparación, pickup y entrega.
Logs	JSON estructurado con <code>tenant</code> , <code>order_id</code> , <code>event_id</code> y <code>severity</code> .
Trazas	Ingreso webhook → comando → DB → evento → notificación.
Alertas	Pago sin pedido, pedido estancado, cola creciente, tracking perdido y webhook inválido.

10.2 Runbooks iniciales

- Pago aprobado sin confirmación del pedido.
- Restaurante no acepta dentro del SLA.
- No hay repartidor o la oferta expira.
- Tracking interrumpido durante el viaje.
- Código incorrecto o cliente ausente.
- Webhook caído, duplicado o fuera de orden.
- Liquidación diaria inconsistente.

11. Escalabilidad y resiliencia

11.1 Patrones obligatorios desde el MVP

- Outbox transaccional para eventos derivados de cambios en base de datos.
- Idempotency keys en checkout, comandos y callbacks.

- Optimistic locking o versionado en pedidos y asignaciones.
- Circuit breakers y timeouts en proveedores externos.
- Colas con reintentos acotados y dead-letter queue.
- Particionado lógico por tenant y archivado de ubicación histórica.

11.2 Cuándo separar servicios

Señal	Candidato a separar
Carga realtime domina recursos	Tracking gateway y almacenamiento de pings.
Mensajería crece o cambia de proveedor	Notification service / channel adapters.
Equipo de pagos requiere ciclo independiente	Payment orchestration y ledger.
Despacho incorpora optimización compleja	Dispatch engine.
Catálogo alimenta varios canales	Catalog service y búsqueda.

ANTI-PATRÓN

No separar microservicios por cada tabla ni por anticipación. La separación debe responder a límites de escala, seguridad, disponibilidad o autonomía de equipo.

12. Estrategia de pruebas

Nivel	Objetivo	Cobertura prioritaria
Unitarias	Reglas puras y transiciones.	Totales, zonas, estados, tarifas y códigos.
Integración	Persistencia y contratos internos.	DB, outbox, jobs, locks y ledger.
Contrato	Compatibilidad con proveedores.	Payloads Meta, Mercado Pago y mapas.
E2E	Flujos nominales y excepciones.	Pedido, pago, cancelación, delivery y soporte.
Carga	Capacidad y degradación.	Webhooks, pings, realtime y workers.
Seguridad	Autorización y abuso.	Tenancy, tokens, rate limit e inyección.

12.1 Datos y sandbox

- No usar datos personales reales en desarrollo.
- Fixtures versionadas por escenarios operativos.
- Reloj controlable para expiraciones y SLA.
- Simuladores de webhooks con duplicados, demora y desorden.
- Pruebas periódicas de restore y replay.

13. MVP técnico y roadmap

Fase	Entrega técnica	Criterio de salida
0. Fundaciones	Repos, CI/CD, tenancy, observabilidad y contratos.	Entorno productivo mínimo y trazabilidad.
1. Pedido asistido	Catálogo, conversación, carrito, pedido y panel.	Pedidos reales sin pago ni delivery automatizado.
2. Pago	Checkout, OAuth, webhooks y conciliación.	Pago aprobado trazable e idempotente.
3. Entrega	Ofertas, PWA, tracking, código e incidencias.	Ciclo completo en zona piloto.
4. Operación	Backoffice, alertas, runbooks y liquidación.	Operación diaria con soporte medible.
5. Optimización	Automatización, promociones y búsqueda.	Mejoras basadas en métricas del piloto.

13.1 Equipo mínimo sugerido

- Tech lead / arquitecto con responsabilidad end-to-end.
- Backend engineer orientado a integraciones y dominio.
- Frontend engineer para paneles y PWA.
- QA automation con foco en flujos y contratos.
- DevOps/Cloud part-time durante fundaciones y hardening.
- Producto/operaciones como dueño de reglas, excepciones y SLA.

14. Decisiones pendientes

1. País y ciudad del piloto; condicionan pagos, privacidad, mapas y soporte.
2. Proveedor final de WhatsApp y modalidad de catálogo.
3. Modelo contractual de Mercado Pago y alcance real del split.
4. Quién recibe y liquida el componente logístico durante el MVP.
5. Frecuencia de tracking compatible con batería, costo y precisión.
6. Retención legal/operativa de mensajes, ubicación y evidencias.
7. Proveedor cloud y servicios administrados aprobados.
8. SLA de cada estado y autoridad para cancelar, reembolsar o forzar cierre.

GATE DE ARQUITECTURA

No comenzar el desarrollo de pagos y liquidaciones hasta cerrar el modelo contractual, el flujo de fondos y el responsable fiscal de cada concepto.

Anexo A. Contratos de eventos

Evento	Productor	Consumidores
order.created	Pedidos	Conversación, métricas
payment.approved	Pagos	Pedidos, notificaciones, ledger
order.accepted	Pedidos	Logística, cliente
order.ready	Restaurante / pedidos	Despacho, repartidor
delivery.assigned	Logística	Repartidor, restaurante, cliente
delivery.location.updated	Tracking	Realtime, operación

Evento	Productor	Consumidores
delivery.completed	Logística	Pedidos, pagos, notificaciones
settlement.generated	Ledger	Operación, repartidor

Envelope mínimo

event_id, event_type, occurred_at, producer, schema_version, tenant_id, correlation_id, aggregate_id, aggregate_version, actor y payload. Los datos sensibles se reducen al mínimo y cada consumidor valida la versión del schema.

Anexo B. API inicial sugerida

Método y recurso	Uso
POST /webhooks/whatsapp	Ingreso de mensajes y estados.
POST /webhooks/payments	Ingreso de eventos de pago.
POST /orders	Crear pedido desde carrito validado.
POST /orders/{id}/transitions	Ejecutar transición autorizada.
POST /orders/{id}/checkout	Emitir checkout idempotente.
POST /deliveries/{id}/offers	Publicar oferta de viaje.
POST /deliveries/{id}/locations	Registrar ping geográfico.
GET /tracking/{token}	Consultar estado y posición autorizada.
POST /deliveries/{id}/complete	Validar código y cerrar entrega.
GET /ops/orders/{id}/timeline	Timeline operativa unificada.

Anexo C. Referencias técnicas oficiales

[Mercado Pago — Split Payments 1:1](#)

[Mercado Pago — Integración marketplace](#)

[Mercado Pago — OAuth](#)

[MDN — Geolocation API](#)

[Meta for Developers — WhatsApp Cloud API](#)

[OpenTelemetry — documentación](#)

NOTA DE VALIDACIÓN

Las capacidades, límites, costos y condiciones comerciales de proveedores externos pueden cambiar. Deben verificarse nuevamente durante el diseño detallado y antes de producción.